

4 · ארבעת סוגי ה-JOIN – אותה שאליתה, תוצאה שונה

נתונים: Dana ו-Avi במחלקה 10 (R&D), Noa בלי מחלקה; מחלקות HR ו-Finance בלי עובדים.

```
SELECT e.first_name, d.dept_name
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id;
-- רק זוגות תואמים: Dana|R&D, Avi|R&D

... LEFT JOIN department d ON e.dept_id = d.dept_id;
-- NULL: + Noa|NULL → כל העובדים; בלי התאמה

... RIGHT JOIN department d ON e.dept_id = d.dept_id;
-- NULL|HR, NULL|Finance + כל המחלקות

SELECT COUNT(*) FROM employee CROSS JOIN department;
-- ON שורות (3×9=9), אין N×M: מכפלה קרטזית
```

שנחתם ON? FROM a JOIN b בלי ON (או FROM a, b WHERE) = מכפלה קרטזית בטעות. **ON לא חייב שוויון:** אפשר a.x > b.y .ON CROSS JOIN עם WHERE מתאים ≡ INNER JOIN.

5 · תבניות JOIN חשובות למבחן

אנטי-JOIN – שורות שמאליות בלי התאמה (עובדים שלא משובצים לאף פרויקט):

```
SELECT e.first_name, e.last_name
FROM employee e
LEFT JOIN works_on w ON e.emp_id = w.emp_id
WHERE w.emp_id IS NULL; -- נשארו רק חסרי-ההתאמה
```

Self-JOIN – טבלה מול עצמה עם שני כינויים (עובד ← שם המנהל שלו):

```
SELECT e.first_name AS worker, m.first_name AS manager
FROM employee e
LEFT JOIN employee m ON e.manager_id = m.emp_id;
-- יופיע (NULL) כדי שגם עובד בלי מנהל
```

USING – קיצור ל-ON כשעמודת החיבור נקראת אותו דבר בשתי הטבלאות:

```
SELECT e.first_name, d.dept_name
FROM employee e JOIN department d USING (dept_id);
-- LEFT JOIN עובד גם עם ; ON e.dept_id = d.dept_id : שקול ל
```

חיבור 3 טבלאות (N:M דרך טבלת קשר) – מי עובד על איזה פרויקט וכמה שעות:

```
SELECT e.first_name, p.proj_name, w.hours
FROM employee e
JOIN works_on w ON e.emp_id = w.emp_id
JOIN project p ON w.proj_id = p.proj_id
WHERE w.hours >= 20;
```

תנאי ב-ON מול WHERE ב-OUTER JOIN: תנאי על הטבלה החיצונית שנכתב ב-WHERE מסנן את שורות ה-NUL – וה-LEFT הופך בפועל ל-INN. תנאי שצריך לחול רק על ההתאמה – לכתוב ב-ON:

```
LEFT JOIN department d
ON e.dept_id = d.dept_id AND d.dept_name = 'R&D'
-- R&D- כל העובדים נשארים; ההתאמה רק ל
```

6 · איחוד תוצאות של שאליות (מהדף-עזר הרשמי)

```
SELECT city FROM employee -- שורות על שורות
UNION
SELECT city FROM suppliers; -- איחוד, בלי כפולים
-- UNION ALL -- איחוד כולל כפולים
-- INTERSECT -- רק שורות שמופיעות בשתי התוצאות
-- MINUS -- שורות של הראשונה שאינן בשנייה
```

שתי השאליות חייבות אותו מספר עמודות וטיפוסים תואמים. FULL OUTER JOIN (מהדף-עזר) = איחוד LEFT+RIGHT – לא קיים ב-MYSQL; ממומש כ-... LEFT JOIN UNION ... RIGHT JOIN.

הסכמה לכל הדוגמאות בדף:

```
department(dept_id PK, dept_name, location)
employee (emp_id PK, first_name, last_name, salary,
         city, age, dept_id FK→department,
         manager_id FK→employee)
project (proj_id PK, proj_name)
works_on (emp_id FK, proj_id FK, hours, PK=(emp_id,proj_id))
```

1 · אנטומיית SELECT – כל החלקים בשאליתה אחת

שולפת לכל עובד ששכרו מעל 5000 את שם המחלקה, שמו ושכרו השנתי, ממוינים מהגבוה לנמוך, שורות 11–15 בלבד:

```
SELECT DISTINCT -- DISTINCT: כפולות
d.dept_name, -- עמודה רגילה (עם כינוי טבלה)
e.first_name AS name, -- כינוי לעמודה (רשות AS)
e.salary * 12 AS yearly_sal -- עמודה מחושבת
FROM employee e -- כינוי טבלה
JOIN department d ON e.dept_id = d.dept_id
WHERE e.salary > 5000 -- סינון שורות
ORDER BY yearly_sal DESC, -- מיון ראשי: יורד
         name ASC -- שובר שוויון: עולה (ברירת מחזל)
LIMIT 5 OFFSET 10; -- LIMIT (או) דלג על 10, החזר 5
10,5)
```

סדר כתיבה: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT.

סדר ביצוע לוגי: FROM(+JOIN) → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. הערות: עד סוף שורה --.

2 · WHERE – כל האופרטורים בדוגמה אחת

כל תנאי כאן הוא אופרטור שנלמד – שימו לב לסוגריים סביב OR:

```
SELECT first_name FROM employee
WHERE (salary BETWEEN 5000 AND 10000 -- טווח, כולל הקצוות
      OR dept_id IN (10, 20, 30)) -- שקול לשרשרת
AND city <> 'Haifa' -- שונה (גם !=)
AND first_name LIKE 'D%' -- מתחיל ב
AND last_name LIKE '_o%' -- סתו כלשהו, אז
AND manager_id IS NOT NULL -- NULL = לעולם לא
AND NOT (age < 25 OR age > 60);
```

אופרטור	משמעות
= < > != < > > =	השוואה; < > = שונה
BETWEEN a AND b	טווח כולל שני הקצוות
IN (v1,v2,...)	שווה לאחד מהרשימה; NOT IN – לאף אחד
LIKE 'תבנית'	% = רצף כלשהו (גם ריק) ; _ = בדיוק תו אחד
IS NULL / IS NOT NULL	בדיקת ערך חסר; NULL = תמיד UNKNOWN
AND OR NOT	לוגיים; AND קודם ל-OR – להוסיף סוגריים!

'%D' LIKE מתחיל ב-D · 'n%' מסתיים ב-n · '%ss%' מכיל 'ss' · '_o_' בדיוק 3 תווים ו-o באמצע · '____' בדיוק 4 תווים. מחרוזות נכתבות בגרש בודד ('Dana'); מספרים – בלי גרש.

3 · תבניות מהמבחן לדוגמה של המרצה

האירועים שהשתתפו בהם יותר מ-5 אנשים (ספירה בטבלת הקשר):

```
SELECT event_id
FROM Connection
GROUP BY event_id
HAVING COUNT(*) > 5;
```

אנשים שהיו ב«יום הסטודנט», או ב«מבחן» וגם גילם מעל 25 – שימו לב שהסוגריים קובעים שה-AND חל רק על תנאי המבחן:

```
SELECT DISTINCT p.f_name, p.l_name
FROM Person p
JOIN Connection c ON p.ID = c.ID
JOIN Event e ON c.event_id = e.event_id
WHERE e.description = 'student day'
      OR (e.description = 'exam' AND p.age > 25);
-- DISTINCT: אדם שהיה בשני האירועים יופיע פעם אחת
```

שלושת המרוויחים הגדולים; LIMIT 3; ORDER BY salary DESC

9 · DDL – הגדרת מבנה + טיפוסים נתונים

משפחות הפקודות: **DDL** = מבנה (CREATE/ALTER/DROP) · **DML** = נתונים (SELECT/INSERT/UPDATE/DELETE) · **DCL** = בקרה והרשאות. יצירת טבלה עם כל הטיפוסים (ההערות = מתי משתמשים):

```
CREATE TABLE product (
  product_id INT PRIMARY KEY AUTO_INCREMENT, -- 1,2,3 רץ...
  name VARCHAR(100) NOT NULL, -- טקסט משתנה עד 100
  sku CHAR(8), -- טקסט באורך קבוע
  descr TEXT, -- טקסט ארוך
  price DECIMAL(8,2), -- מדויק - לכסף (8 ספרות, 2
  -- אחרים הנקודה)
  weight FLOAT, -- קירוב - לא לכסף!
  qty INT, big_id BIGINT,
  in_stock BOOLEAN DEFAULT TRUE,
  added_on DATE, updated_at DATETIME -- וגם TIMESTAMP
);

-- המספרים בטיפוסים:
-- DECIMAL(p,s): מה p עד s ספרות בסך הכול, מה p עד s ספרות בסך הכול,
-- DECIMAL(8,2) → 999999.99 ; מקסימום DECIMAL(5,1) → 9999.9
-- VARCHAR(n): n עד (תופס לפי האורך בפועל)
-- CHAR(n): n בדיוק (קצר יותר - מרופד ברווחים)

CREATE DATABASE shop; USE shop; SHOW TABLES; -- ניהול מסד
DESCRIBE product; -- מונה הטבלה (= SHOW COLUMNS FROM product)
ALTER TABLE product ADD email VARCHAR(100); -- הוספת עמודה
ALTER TABLE product MODIFY name VARCHAR(150); -- שינוי טיפוס
ALTER TABLE product DROP email; -- מחיקת עמודה
ALTER TABLE product RENAME TO item; -- שינוי שם טבלה
ALTER TABLE item RENAME COLUMN sku TO code; -- שינוי שם עמודה
ALTER TABLE item ADD CHECK (price > 0); -- הוספת אילוץ
ALTER TABLE item DROP CONSTRAINT c; -- הסרת אילוץ
CREATE TABLE rich AS -- טבלה חדשה משאילתה
  SELECT * FROM employee WHERE salary > 10000;
-- ולשנות סדר/שמות עמודות בכינויים LIMIT אפשר גם עם
DROP TABLE rich; -- מחיקת הטבלה כולה
```

אינדקסים – מבנה עזר לזירוז חיפוש על עמודות (במחיר עדכונים איטיים יותר):

```
CREATE INDEX idx_city ON employee(city); -- אינדקס רגיל
CREATE UNIQUE INDEX idx_ssn ON employee(emp_ssn); -- גם אוכף
-- ייחודיות
DROP INDEX idx_city ON employee;
```

10 · DML – הוספה, עדכון, מחיקה

```
INSERT INTO employee (emp_id, first_name, salary)
VALUES (1, 'Dana', 8000); -- שורה אחת

INSERT INTO employee (first_name, salary)
VALUES ('Noa', 9000), ('Avi', 6500); -- כמה שורות PK מ-AUTO_INCREMENT

INSERT INTO managers -- הוספה מתוך שאילתה
  SELECT * FROM employee WHERE manager_id IS NULL;

-- בלי ערך ← נכשל NOT NULL; DEFAULT; NULL עמודות שלא צוינו
UPDATE employee
SET salary = salary * 1.1, -- אפשר כמה השמות בפסיק
  city = 'Ramat Gan'
WHERE dept_id = 10; -- כולם - WHERE בלי
-- מתעדכנים!
UPDATE t SET x = (SELECT MIN(x) FROM s); -- עדכון מתת-שאילתה
UPDATE city SET code = CONCAT(city_id, '_', country_id); -- חיבור מחרוזות

DELETE FROM employee WHERE dept_id IS NULL;
DELETE FROM t WHERE id NOT IN (SELECT id FROM keep);
```

מה עושה	סוג	פקודה
מחקת שורות (אפשר חלקית); ניתנת לביטול בטרנזקציה	DML	DELETE FROM t [WHERE]
מרוקנת הכול מהר + מאפסת AUTO_INCREMENT	DDL	TRUNCATE TABLE t
מחקת את הטבלה עצמה (מבנה + נתונים)	DDL	DROP TABLE t

זהירות: DELETE / UPDATE בלי WHERE פוגעים בכל השורות.

7 · אגרגציה + GROUP BY + HAVING – הכול בשאילתה אחת

לכל מחלקה בתל-אביב עם יותר מ-2 עובדים: כמה עובדים, כמה עם שכר, וסטטיסטיקות שכר:

```
SELECT d.dept_name,
  COUNT(*) AS num_rows, -- סופר שורות (גם NULL)
  COUNT(e.salary) AS with_sal, -- רק ערכים לא NULL
  AVG(e.salary) AS avg_sal, -- בממוצע NULL-מתעלם מ
  SUM(e.salary) AS total,
  MAX(e.salary) AS top, MIN(e.salary) AS bottom
FROM department d
LEFT JOIN employee e ON e.dept_id = d.dept_id
WHERE d.location = 'Tel Aviv' -- סינון שורות, לפני קיבוץ
GROUP BY d.dept_id, d.dept_name -- קבוצה לכל מחלקה
HAVING COUNT(*) > 2 -- סינון קבוצות, אחרי קיבוץ
ORDER BY avg_sal DESC;
```

הכלל: כל עמודה ב-SELECT שאינה בתוך פונקציית אגרגציה – חייבת להופיע ב-WHERE .GROUP BY מסנן שורות ואסור לו אגרגציה; **HAVING** מסנן קבוצות ומותר לו (HAVING AVG(salary)>9000).

אם שכר של עובד אחד NULL מתוך 4: COUNT(*)=4, COUNT(salary)=3, AVG=3/4 (!).

```
SELECT AVG(salary) FROM employee; -- GROUP BY בלי:
-- האגרגציה על כל הטבלה – מוחזרת שורה אחת
SELECT dept_id, city, COUNT(*) -- קיבוץ לפי צירוף
FROM employee GROUP BY dept_id, city; -- (מחלקה, עיר)
SELECT COUNT(DISTINCT city) FROM employee; -- ספירת ערכים שונים
```

8 · תתי-שאליות (Subqueries)

סקלרית (ערך יחיד) – מי מרוויח מעל הממוצע הכללי:

```
SELECT first_name FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);
```

רשימה עם IN – עובדי המחלקות שבתל-אביב:

```
SELECT first_name FROM employee
WHERE dept_id IN (SELECT dept_id FROM department
  WHERE location = 'Tel Aviv');
```

מתואמת (correlated) – המרוויח הכי הרבה בכל מחלקה (הפנימית רצה לכל שורה חיצונית):

```
SELECT d.dept_name, e.first_name, e.salary
FROM employee e JOIN department d ON e.dept_id = d.dept_id
WHERE e.salary = (SELECT MAX(e2.salary) FROM employee e2
  WHERE e2.dept_id = e.dept_id);
```

בתוך SELECT – ערך מחושב לכל שורה (כמה פרויקטים לכל עובד):

```
SELECT e.first_name,
  (SELECT COUNT(*) FROM works_on w
  WHERE w.emp_id = e.emp_id) AS num_projects
FROM employee e;
```

בתוך FROM – טבלה זמנית עם כינוי:

```
SELECT t.dept_id, t.avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal
  FROM employee GROUP BY dept_id) t
WHERE t.avg_sal > 9000;
```

מלכודת: (NULL תת-שאילתה שמחזירה) NOT IN לא מחזיר אף שורה! להשתמש באנטי-JOIN (סעיף 4) או לסנן: WHERE x NOT IN (SELECT y FROM t WHERE y IS NOT NULL)

14 · טריגרים (Triggers)

קוד שרץ אוטומטית על אירוע: INSERT/UPDATE/DELETE × BEFORE/AFTER, לכל שורה (FOR EACH ROW; FOR EACH STATEMENT – פעם אחת לכל הפקודה). NEW=השורה החדשה, OLD=הישנה:

```
CREATE TRIGGER log_new_emp
AFTER INSERT ON employee FOR EACH ROW
BEGIN
    INSERT INTO emp_log VALUES (NEW.first_name, NOW());
END;

CREATE TRIGGER log_raise -- תיעוד שינוי שכר בלבד
AFTER UPDATE ON employee FOR EACH ROW
BEGIN
    IF (OLD.salary != NEW.salary) THEN
        INSERT INTO pay_log VALUES (OLD.emp_id, OLD.salary,
NEW.salary);
    END IF;
END;

CREATE TRIGGER clean_name -- BEFORE לשנות את NEW
BEFORE INSERT ON employee FOR EACH ROW
SET NEW.first_name = UPPER(NEW.first_name);

CREATE TRIGGER archive_emp -- AFTER DELETE: יש רק OLD
AFTER DELETE ON employee FOR EACH ROW
INSERT INTO emp_archive VALUES (OLD.emp_id, OLD.first_name);

DROP TRIGGER log_new_emp; -- מחיקת טריגר
SHOW TRIGGERS; -- רשימת הטריגרים
```

זמינות: INSERT – רק UPDATE · NEW – שניהם · DELETE – רק OLD. גוף של פקודה בודדת – בלי BEGIN...END. קיימים 6 שילובים – {INSERT,UPDATE,DELETE} × {BEFORE,AFTER} – כולם באותו תחביר. ב-BEFORE UPDATE אפשר «לתקן» ערך: IF (NEW.x < OLD.x) THEN בדף-העזר מופיעה גם צורה כללית (מסדים אחרים): טריגר שמפעיל פרוצדורה – EXECUTE stored_procedure.

BEFORE – לתקן/לאמת את NEW לפני שמירה; **AFTER** – לוגים ועדכון טבלאות אחרות.

15 · פונקציות ופרוצדורות מאוחסנות

פונקציה – מקבלת פרמטרים, **חייבת** להחזיר ערך יחיד (RETURNS), ונקראת בתוך שאלתות:

```
CREATE FUNCTION actor_count_fn(in_fn VARCHAR(45),
                              in_ln VARCHAR(45))
RETURNS INTEGER
DETERMINISTIC -- אותו קלט תמיד מחזיר אותו פלט
BEGIN
    DECLARE a_count INTEGER DEFAULT 0; -- משתנה מקומי
    SELECT COUNT(*) INTO a_count -- השאילתה: שורה אחת בלבד
    FROM actor a JOIN film_actor fa ON a.actor_id=fa.actor_id
    WHERE a.first_name = in_fn AND a.last_name = in_ln;
    -- או השמה ישירה: SET a_count = 5;
    RETURN a_count;
END;

CREATE FUNCTION annual(m DECIMAL(10,2)) -- פונקציית ביטוי
RETURNS DECIMAL(10,2) DETERMINISTIC RETURN m * 12;

SELECT actor_count_fn('NICK','WAHLBERG'); -- ערך בודד
SELECT first_name, actor_count_fn(first_name,last_name)
FROM actor WHERE actor_count_fn(first_name,last_name) > 20;
```

פרוצדורה – מופעלת ב-CALL, מחזירה דרך פרמטרי OUT (או SELECT), ויכולה לשנות נתונים:

```
CREATE PROCEDURE dept_summary(IN in_dept INT,
                              OUT cnt INT, OUT total INT)
BEGIN
    SELECT COUNT(*), SUM(salary) INTO cnt, total
    FROM employee WHERE dept_id = in_dept;
    -- רגיל בגוף – התוצאה תוצג לקורא SELECT אפשר גם
    -- פרוצדורה רשאית לשנות נתונים – INSERT/UPDATE וגם
END;
CALL dept_summary(10, @c, @t); SELECT @c, @t;

DROP FUNCTION annual; DROP PROCEDURE dept_summary;
```

ההבדל: פונקציה = ערך יחיד, בתוך SELECT, קלט בלבד; פרוצדורה = CALL, פרמטרי IN/OUT/INOUT, כמה פעולות, יכולה לעדכן טבלאות.

11 · כל האילוצים בטבלה אחת

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY, -- ייחודי + NOT NULL
    emp_ssn CHAR(9) NOT NULL UNIQUE, -- מפתח מועמד
    emp_name VARCHAR(40) NOT NULL, -- חובה, לא ייחודי
    status VARCHAR(10) DEFAULT 'active', -- אם לא סופק ערך
    pay_rate DECIMAL(5,2) NOT NULL,
    dept_id INT,
    manager_id INT, -- מפתח זר לעצמה!
    CHECK (pay_rate > 5.25), -- תנאי על כל שורה
    CHECK (status IN ('active','vacation','suspended')),
    FOREIGN KEY (dept_id) REFERENCES department(dept_id),
    FOREIGN KEY (manager_id) REFERENCES employee(emp_id)
);
-- מפתח ראשי מורכב – חייב אחרי העמודות:
CREATE TABLE works_on (
    emp_id INT, proj_id INT, hours DECIMAL(5,1),
    PRIMARY KEY (emp_id, proj_id),
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id),
    FOREIGN KEY (proj_id) REFERENCES project(proj_id)
);
-- שמפר אילוץ – נדחה עם שגיאה:
INSERT INTO employee VALUES (1,'123456789','Dana','active',3.0,...);
-- ERROR: CHECK violated (3.0 < 5.25)
```

אילוץ	מבטיח
PRIMARY KEY	ייחודי + לא-NULL; אחד לטבלה (אפשר מורכב)
UNIQUE	ייחודיות; NULL מותר; אפשר כמה בטבלה
NOT NULL	חובה ערך
DEFAULT v	ערך כשלא סופק
CHECK (cond)	התנאי לא-FALSE לכל שורה; NULL עובר (UNKNOWN)
FOREIGN KEY	שלמות התייחסות – הערך קיים בטבלת האב

אילוצים ברמת הטבלה (אחרי העמודות) – גם על צירופי עמודות:

```
CREATE TABLE t (
    c1 INT, c2 INT, c3 INT,
    UNIQUE (c2, c3), -- ייחודי (c2,c3) הצירוף
    CHECK (c1 > 0 AND c1 >= c2) -- תנאי בין עמודות
);
```

12 · מפתח זר: ON DELETE / ON UPDATE

```
CREATE TABLE health_plan (
    emp_ssn CHAR(9) PRIMARY KEY,
    provider VARCHAR(20) NOT NULL,
    FOREIGN KEY (emp_ssn) REFERENCES employee(emp_ssn)
    ON DELETE CASCADE -- מחיקת האב מוחקת את הבנים
    ON UPDATE CASCADE -- עדכון מפתח האב מתעדכן אצל הבנים
);
```

- **CASCADE** – הפעולה «מדרדרת» לשורות המפנות.
- **SET NULL** – המפתח הזר אצל הבנים הופך NULL.
- **RESTRICT / NO ACTION** (ברירת מחדל) – הפעולה על האב נכשלת כל עוד יש בנים מפנים.

הפרות FK: הוספה/עדכון של בן עם ערך שלא קיים באב – נדחים תמיד.

13 · Views – טבלה וירטואלית

שאילתה שמורה שמתנהגת כטבלה; לא מאחסנת נתונים (רצה בכל פנייה). למה: פישוט, אבטחה (חשיפת עמודות מסוימות), שימוש חוזר:

```
CREATE VIEW actor_film_vw AS
SELECT a.first_name, a.last_name, f.title, f.film_id
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON f.film_id = fa.film_id;

SELECT * FROM actor_film_vw; -- כמו טבלה רגילה
SELECT * FROM actor_film_vw v -- JOIN וגם ב-WHERE
JOIN category c ON ... WHERE v.title LIKE 'A%';

CREATE VIEW v2 (name, total) AS SELECT ...; -- שמות עמודות ל-View
DROP VIEW actor_film_vw; -- מחיקת View
-- CREATE TEMPORARY VIEW, WITH CHECK OPTION קיימים גם
```

16 · המודל הרלציוני – מושגי יסוד

- מודלים:** היררכי (עץ, הורה אחד) → רשת (כמה קשרים) → **רלציוני** (טבלאות, גישה ישירה, מבוסס אלגברת קבוצות).
- רלציה**=טבלה; **סכמה**=שם+עמודות+אילוצים (קבועה); **מופע**=תוכן הטבלה כרגע (משתנה).
- מפתח ראשי (PK)** – מזהה ייחודי, לא-NULL. **מפתח מועמד** – עמודה ייחודית שלא נבחרה כ-PK (מסומנת UNIQUE).
- מפתח זר (FK)** – עמודה שמפנה ל-PK של טבלה אחרת (או של אותה טבלה).
- NULL** – «לא ידוע»; מותר אלא אם הוגדר NOT NULL; אסור ב-PK.
- שלמות ישויות:** לכל טבלה PK ייחודי ולא-NULL. **שלמות התייחסות:** כל ערך FK קיים בטבלה המופנית – ה-DBMS אוכף את שתיהן.
- מפתח-על (super key):** קבוצת עמודות שמזהה שורה ייחודית; PK = מפתח-על מינימלי (בלי עמודות מיותרות). משמש בהגדרת BCNF.
- DBMS** – התוכנה שמנהלת את המסד (MySQL): אחסון, שאליות, אכיפת אילוצים, משתמשים במקביל, הרשאות וגיבוי.
- למה לא סתם קבצים?** כפילות וחוסר-עקביות, תלות של התוכנית במבנה הקובץ, אין שאליות/אילוצים, אין גישה מקבילה ואבטחה.
- נתונים מול מידע:** Data = ערכים גולמיים בלי הקשר; Information = נתונים בהקשר/אחרי עיבוד.
- עיבוד שאלית ב-DBMS:** ניתוח (parser) → אופטימיזציה (בחירת execution plan) → הרצה (execution engine).
- רלציוני מול NoSQL:** רלציוני = סכמה קבועה, SQL, ACID; NoSQL = (מסמכים/מפתח-ערך) = סכמה גמישה, סקלאביליות – נלמד בקורס רק כרקע.

17 · מידול ER – סימון ומושגים

סימון בדיאגרמה	משמעות
מלבן	ישות (Entity) – עצם בעולם (עובד, מחלקה)
אליפסה	תכונה: קו תחתון = תכונת מפתח
מעוין	קשר (Relationship) בין ישויות
אליפסה כפולה	תכונה רב-ערכית (כמה טלפונים)
אליפסה מקווקוות	תכונה נגזרת (גיל ← תאריך לידה)
מלבן כפול	ישות חלשה – מזוהה רק עם מפתח האב
מעוין כפול	הקשר המזהה שמחבר ישות חלשה לאב שלה
N / M על הקו	קרדינליות הקשר

- תכונה מורכבת:** כתובת = רחוב+עיר+מיקוד. **פשוטה:** אטומית.
- קרדינליות:** 1:1 (עובד→כרטיס) · N:1 (מחלקה→עובדים) · M:N (סטודנטים→קורסים).
- דרגת קשר:** רוב הקשרים בינאריים (2 ישויות); קיים גם קשר משולש (ternary).
- קשר וקורסיבי:** ישות אל עצמה – עובד «עובד תחת» מנהל.

18 · המרת ER למודל רלציוני – כללי ההמרה

- ישות חזקה** → **טבלה**; תכונת המפתח → PK.
- תכונה מורכבת** «משתטחת» לעמודות: `customer(cust_id, street, city, zip)`.
- תכונה רב-ערכית** → **טבלה נפרדת** עם FK חזרה: `cust_addr(cust_id FK, cust_addr(cust_id FK, address))`.
- קשר N:1** – מפתח הצד ה«אחד» נכנס כ-FK בצד ה«רבים»: `employee(..., dept_id FK)`. לעולם לא הפוך!
- קשר 1:1** – FK באחד הצדדים (עדיף בצד עם השתתפות מלאה; NULL אם חלקית) או מיזוג לטבלה אחת.
- קשר M:N** → **טבלת קשר חדשה:** שני FK, PK = הצירוף, ותכונות הקשר יושבות בה: `borrower(cust_id FK, loan_id FK, access_date) PK=(cust_id,loan_id)`
- ישות חלשה** → **טבלה** עם PK מורכב (FK של האב + המפתח החלקי), ולרוב `ON dependent(emp_id FK, dep_name) PK=:DELETE CASCADE (emp_id,dep_name)`
- קשר וקורסיבי** – FK לאותה טבלה: `employee(emp_id PK, ..., manager_id FK→employee)`.
- המפתח של טבלת-קשר** (כשממפים קשר): M:N → צירוף מפתחות שני הצדדים; N:1 → מפתח צד ה«רבים»; 1:1 → אחד מהמפתחות.
- קשר משולש (n-ary):** טבלת קשר עם FK לכל ישות משתתפת; המפתח לפי אותם כללים.
- מיזוג סכמות ב-1:1:** אפשר לאחד לטבלה אחת (חוסך JOIN) – במחיר NULL-ים כשהשתתפות חלקית.

19 · נירמול – הגדרות + הפירוק המלא מהשיעור

תלות פונקציונלית X→Y: ערך X קובע חד-משמעית את Y. **אנומליות** בטבלה לא-מנורמלת: הוספה (אין קורס בלי סטודנט), עדכון (שם קורס בהרבה שורות), מחיקה (סטודנט אחרון מוחק את הקורס).

צורה	דרישה
1NF	ערכים אטומיים, אין קבוצות חוזרות, יש PK
2NF	1NF + אין תלות חלקית : כל עמודה תלויה ב כל המפתח (רלוונטי למפתח מורכב)
3NF	2NF + אין תלות מעבר : עמודה תלויה במפתח בלבד, לא בעמודה לא-מפתח
BCNF	לכל תלות X→Y, ה-X מפתח-על

הדוגמה מהשיעור – `Student_Report(StudentNo, StudentName, Major, CourseNo, CourseName, InstructorNo, InstructorName, Grade)`

הפרדת הקבוצה החוזרת (הקורסים) + זיהוי מפתחות: 1NF-- Student(StudentNo PK, StudentName, Major) StudentCourse(StudentNo, CourseNo, CourseName, InstructorNo, InstructorName, Grade) PK=(StudentNo,CourseNo) חלק מהמפתח CourseNo והמרצה תלויים רק ב Student: 2NF-- Student(StudentNo PK, StudentName, Major) CourseGrade(StudentNo, CourseNo, Grade) PK=(StudentNo,CourseNo) CourseInstructor(CourseNo PK, CourseName, InstructorNo, InstructorName) תלות מעבר InstructorNo תלוי ב InstructorName: 3NF-- Course(CourseNo PK, CourseName, InstructorNo FK) Instructor(InstructorNo PK, InstructorName) BCNF-- Bor_Loan(CustomerID, LoanID, Amount), איננו מפתח-על ⇒ מפרקים LoanID-וה LoanID→Amount התלות: Loan(LoanID PK, Amount) Bor_Loan(CustomerID, LoanID)
--

כלל אצבע: «כל תכונה תלויה במפתח, בכל המפתח, ורק במפתח». ערכים שחוזרים בעמודות לא-מפתח = סימן לפירוק. זה בדיוק פורמט שאלת המבחן: טבלה גדולה → פירוק + ERD.

20 · מלכודות – צ'ק-ליסט אחרון לפני הגשה

מלכודת	הנכון
NULL = לא תופס כלום	IS NULL / IS NOT NULL
COUNT(col) מדלג על NULL	לספירת שורות: COUNT(*)
AVG מחלק רק בלא-NULL	לב לחישובי ממוצע עם NULL
אגרגציה ב-WHERE – שגיאה	סינון קבוצות רק ב-HAVING
עמודה חופשית ב-SELECT עם GROUP BY	להוסיף אותה ל-GROUP BY
JOIN בלי ON	מכפלה קרטזית – לבדוק שיש ON
תנאי חיצוני ב-WHERE הופך ל-LEFT INNER	לשים את התנאי ב-ON
NOT IN מול 0 – NULL שורות	אנטי-JOIN: LEFT JOIN ... IS NULL
UNIQUE חוסם NULL? לא!	להוסיף גם NOT NULL
CHECK חוסם NULL? לא! (UNKNOWN)	להוסיף גם NOT NULL
UPDATE/DELETE בלי WHERE	פוגע בכל השורות – לוודא WHERE
FK בצד ה«אחד» ב-N:1	FK תמיד בצד ה«רבים»
BETWEEN לא כולל קצוות?	כולל! 5 ≤ 5,10 BETWEEN 5 AND 10
AND/OR בלי סוגריים	AND קודם – לשים סוגריים סביב OR

מתכון לשאלת «כתבו שאלית»: ① אילו טבלאות מחזיקות את הנתונים? ② איך הן מתחברות (FK=PK, איזה JOIN – צריך גם חסרי-התאמה? LEFT) ③ סינון שורות → WHERE ④ «לכל X GROUP BY X = «...X; תנאי על הקבוצה → HAVING ⑤ מה מציגים (SELECT, כינויים) ⑥ מיון/כמות → ORDER BY, LIMIT ⑦ בדיקה: NULL? כפילויות (DISTINCT)? סוגריים ב-OR?